

Design Document — Sales Analytics Platform

Project: Data Engineering Challenge — Olist Sales Analytics

Table of Contents

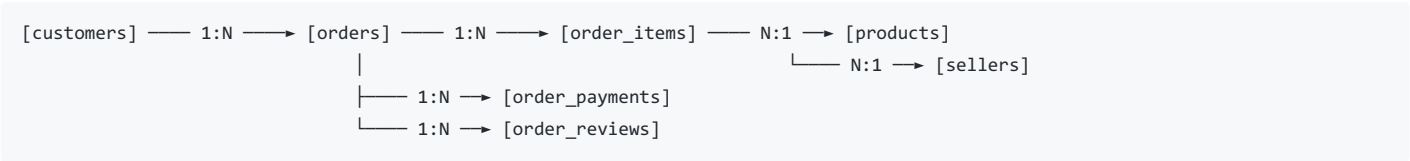
- 1. Business Ambiguities & Decisions
- 2. Data Exploration & Constraints
 - Data Traps & Solutions
- 3. Data Verification Queries
 - Customer Identity Verification
 - Fan-out Verification
 - Items vs Payments Reconciliation
 - Orphan Records & Translation Check
 - Raw vs Warehouse Reconciliation
- 4. System Architecture
 - Three-Tier ELT Pipeline Flow
 - Technology Selection
 - Orchestration & Scheduling Strategy
 - Late-Arriving Dimensions Handling
- 5. Data Model (Star Schema)
 - Fact Tables
 - Dimension Tables
- 6. Ingest & Idempotency Strategy
- 7. Data Quality
 - Automated Tests (dbt Tests)
 - Error Handling & Alerting
- 8. BI Queries
 - Total Revenue & MTD (USD)
 - Revenue by Product Category
 - Revenue by Customer State (Region)
 - Freight Contribution Ratio
 - Payment Methods & Installment Aggregation
 - Lifetime Repeat Buyer Rate
 - 90-Day Repeat Buyer Rate
- 9. Design Dilemmas, Uncertainties & Limitations
 - Design Dilemmas
 - Data Uncertainties
 - Project Limitations
- 10. Scale-up Plan (Handling Millions of Rows)
- 11. Appendix — Assumption Registry

1. Business Ambiguities & Decisions

Issue	Details & Risks	Design Decision & Rationale
Revenue Definition	• Two sources: <code>price (order_items)</code> vs <code>payment_value (order_payments)</code>. • <code>payment_value</code> is at the order level, so it cannot be split by product category. • <code>order_items</code> stores item-level details, enabling categorization.	• Revenue = <code>SUM(order_items.price)</code> (Net product sales, excluding shipping costs <code>freight_value</code>). • Includes orders in <code>approved</code> status (paid but not yet shipped) to reflect real-time sales team performance.
Freight Handling	• Freight is a pass-through shipping cost collected from buyers to pay logistics providers.	• Kept as a separate <code>freight_value</code> metric in the Fact table for shipping cost analysis, and excluded from Revenue.
Tax & Installments	• Tax rates are not explicitly provided in the dataset. • Customers pay in installments (<code>payment_installments > 1</code>).	• Reports Gross Sales (including ICMS consumption tax, which is embedded in the product price). • Assumes Olist receives 100% of the cash flow upfront $\rightarrow$ Records 100% of revenue at order date instead of spreading it over installment periods.
Exchange Rate Date (Order Date)	• Orders have 5 different timestamps.	• Uses <code>order_purchase_timestamp</code> as the baseline date for exchange rates to capture the currency value at the exact time of the customer's purchase decision.
Canceled Orders	• Canceled or unavailable orders distort actual sales figures.	• Categorizes order statuses: - <i>Counted as Revenue</i>: <code>approved</code>, <code>processing</code>, <code>invoiced</code>, <code>shipped</code>, <code>delivered</code>. - <i>Excluded</i>: <code>canceled</code>, <code>unavailable</code>, <code>created</code>. • Maintains the purchase-date exchange rate for canceled orders to

Issue	Details & Risks	ensure balanced reconciliation upon cancellation. Design Decision & Rationale
Precision of Financial Data	• Using float types causes rounding errors during large-scale aggregations.	• Casts all monetary values to <code>DECIMAL(18, 4)</code> at the Staging layer to guarantee financial accounting precision.
Historical MTD Reporting	• Since this is a historical static dataset, using the current system date for MTD calculations results in empty metrics.	• Parameterizes the report date (<code>execution_date</code>) to act as a simulated report date (e.g., <code>2018-08-15</code>).
Repeat Buyer Rate	• No standard definition of a "returning" customer. • Lifetime metrics naturally accumulate over time, masking short-term retention trends. • Purchase cycles vary heavily by category (e.g., FMCG is 1-3 months, major appliances is 1-2 years).	• Lifetime Repeat Buyer Rate: Customers with ≥ 2 lifetime orders (provides an overall cumulative view). • 90-Day Repeat Rate: Customers who placed a follow-up order within 90 days of their previous purchase. This serves as a general baseline for re-engagement campaigns.

2. Data Exploration & Constraints



2.1. Data Traps & Solutions

- Two-Tier Customer Identifiers:**
 - Trap: `customer_id` changes for every order placed.
 - Solution: Use `customer_unique_id` to uniquely identify the customer across multiple purchases and calculate repeat buyer rates.
- Geographical Shift:**
 - Trap: Customers changing their addresses updates their current state, distorting historical revenue-by-region reports.
 - Solution: Apply **SCD Type 2** via **dbt Snapshots** for `customer_state`. This ensures that past orders are mapped to the customer's location at the time of purchase, rather than retroactively shifting historical revenue to their new address.
- Row Inflation (Fan-out Trap):**
 - Trap: Direct JOINS between `order_items` (1-N) and `order_payments` (1-N) multiply records, artificially inflating sales numbers.
 - Solution: Model them as **2 independent Fact tables**: `fact_order_items` and `fact_order_payments`.
- Geolocation Fan-out:**
 - Trap: `zip_code_prefix` in `geolocation` is not unique, containing multiple GPS coordinates for a single zip code.
 - Solution: Exclude this table from the Fact star model. Use the pre-existing `customer_state` in the Customer dimension for regional reports.
- Product Category Translation:**
 - Trap: Some Portuguese category names are missing from the English translation dictionary, leading to lost rows on INNER JOINS.
 - Solution: Use a `LEFT JOIN` combined with `COALESCE(translation, Portuguese_name)`.
- Timezone Discrepancies in Exchange Rates:**
 - Trap: Olist timestamps are in Brazil time (BRT, UTC-3), while the Frankfurter API publishes rates in UTC. Simply casting the timestamp to a date without timezone conversion will cause late-night orders (e.g., 22:00 BRT) to map to the next day's exchange rate.
 - Solution: Standardize timestamps to UTC (`AT TIME ZONE 'America/Sao_Paulo' AT TIME ZONE 'UTC'`) at the Staging layer (`stg_orders`) before extracting the date for exchange rate JOINS.
- Orphan Records:**
 - Trap: Orders referencing `product_id`s that do not exist in the products catalog.
 - Solution: Use `LEFT JOIN` and map missing products to an `'unknown'` category to preserve 100% of sales revenue.
- Weekend FX Rates:**
 - Trap: The Frankfurter API does not publish rates on weekends or market holidays.
 - Solution: Fetch rates using the range API (`GET https://api.frankfurter.dev/v2/2016-09-01..2018-10-31?base=BRL&symbols=USD`) and execute an SQL forward-fill query to **fallback to the last available Friday exchange rate** for weekend dates.

3. Data Verification Queries

These queries run directly on the raw data schema to inspect anomalies:

3.1. Customer Identity Verification

```

-- Check uniqueness of IDs
SELECT
    COUNT(customer_id)                AS total_rows,
    COUNT(DISTINCT customer_id)        AS unique_customer_ids,
    COUNT(DISTINCT customer_unique_id) AS unique_real_customers
FROM customers;

-- Find customers with multiple zip codes
SELECT customer_unique_id, COUNT(DISTINCT customer_zip_code_prefix) AS zip_count
FROM customers
GROUP BY 1 HAVING COUNT(DISTINCT customer_zip_code_prefix) > 1
ORDER BY 2 DESC LIMIT 10;

```

3.2. Fan-out Verification

```

-- Compare raw join rows against base rows
SELECT COUNT(*) AS joined_rows
FROM order_items oi
JOIN order_payments op ON oi.order_id = op.order_id;

SELECT COUNT(*) FROM order_items; -- If joined_rows > count, fan-out occurs.

```

3.3. Items vs Payments Reconciliation

```

WITH items_sum AS (
    SELECT order_id, SUM(price + freight_value) AS items_total
    FROM order_items GROUP BY 1
),
payments_sum AS (
    SELECT order_id, SUM(payment_value) AS payments_total
    FROM order_payments GROUP BY 1
)
SELECT
    COUNT(*)                AS total_orders_compared,
    COUNT(CASE WHEN ABS(i.items_total - p.payments_total) > 0.01 THEN 1 END) AS orders_with_diff,
    AVG(ABS(i.items_total - p.payments_total)) AS avg_diff_br1,
    COUNT(CASE WHEN ABS(i.items_total - p.payments_total) > 0.01 THEN 1 END)::FLOAT
    / NULLIF(COUNT(*), 0) * 100 AS pct_orders_with_diff
FROM items_sum i
JOIN payments_sum p ON i.order_id = p.order_id;

```

3.4. Orphan Records & Translation Check

```

-- Count orphan products
SELECT COUNT(DISTINCT product_id) AS orphan_product_count
FROM order_items
WHERE product_id NOT IN (SELECT product_id FROM products);

-- Identify untranslated categories
SELECT DISTINCT p.product_category_name
FROM products p
WHERE p.product_category_name IS NOT NULL
AND p.product_category_name NOT IN (SELECT product_category_name FROM product_category_name_translation);

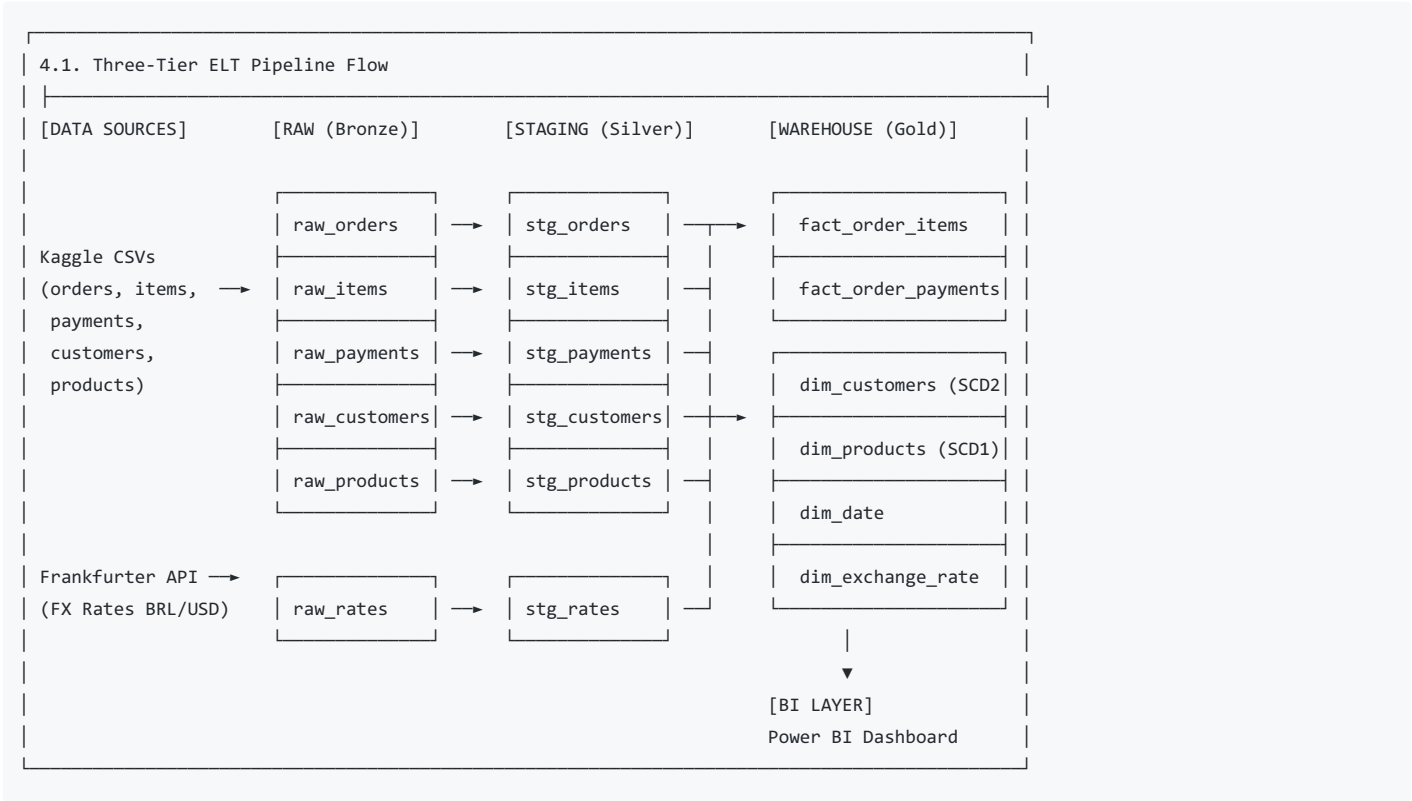
```

3.5. Raw vs Warehouse Reconciliation

```
SELECT
    SUM(r.price)          AS raw_total_br1,
    SUM(f.price_br1)      AS warehouse_total_br1,
    ABS(SUM(r.price) - SUM(f.price_br1)) / NULLIF(SUM(r.price), 0) * 100 AS diff_pct
FROM raw_order_items r
FULL OUTER JOIN fact_order_items f ON r.order_id = f.order_id AND r.order_item_id = f.order_item_id;
```

4. System Architecture

4.1. Three-Tier ELT Pipeline Flow



- **Raw:** Stores 100% raw source files (Kaggle CSVs and Frankfurter API rates).
- **Staging:** Performs data cleaning, type casting (DECIMAL(18, 4)), and handles missing values.
- **Core:** Implements the Star Schema optimized for downstream BI tools.

4.2. Technology Selection

- **Database:** PostgreSQL (Standard SQL dialect with dbt native support).
- **Transformation:** dbt (Orchestrates dependencies, SQL generation, and automated testing).
- **Ingestion:** Python (Fetches APIs, copies CSVs).
- **Orchestration:** Apache Airflow.

4.3. Orchestration & Scheduling Strategy

Airflow DAGs handle initial and daily runs differently:

Scenario 1: Initial Load / Full Backfill Since the raw Kaggle dataset is static, the first execution fetches the entire historical currency exchange rates using the range API, and dbt builds the entire schema from scratch.

Scenario 2: Daily Incremental Refresh Managed via 2 independent Airflow DAGs utilizing Airflow's execution_date parameters:

- **DAG A (dag_refresh_dimensions):** Runs once daily at 00:00 UTC .
 - *Workflow:* Ingests daily master data $\rightarrow$ dbt run Dim $\rightarrow$ dbt test Dim .
 - The dim_customers (SCD Type 2) model utilizes dbt snapshots to track customer updates. The dim_products (SCD Type 1) model uses materialized='table' to overwrite values with the latest status.
- **DAG B (dag_refresh_facts):** Runs 3 times daily at 00:30 , 08:30 , and 16:30 UTC .
 - *Workflow:* Fetches current exchange rate $\rightarrow$ dbt run Fact (incremental merge) $\rightarrow$ dbt test Fact .
 - The incremental merge strategy uses unique keys (order_item_key / order_payment_key) to ensure idempotency.

Self-Healing / Resilience Design: If the system experiences downtime, it self-heals upon restart:

- **Facts:** Employs a MAX(purchase_date) - 30 days sliding lookback window to scan historical dates, capture missing records, and merge them without

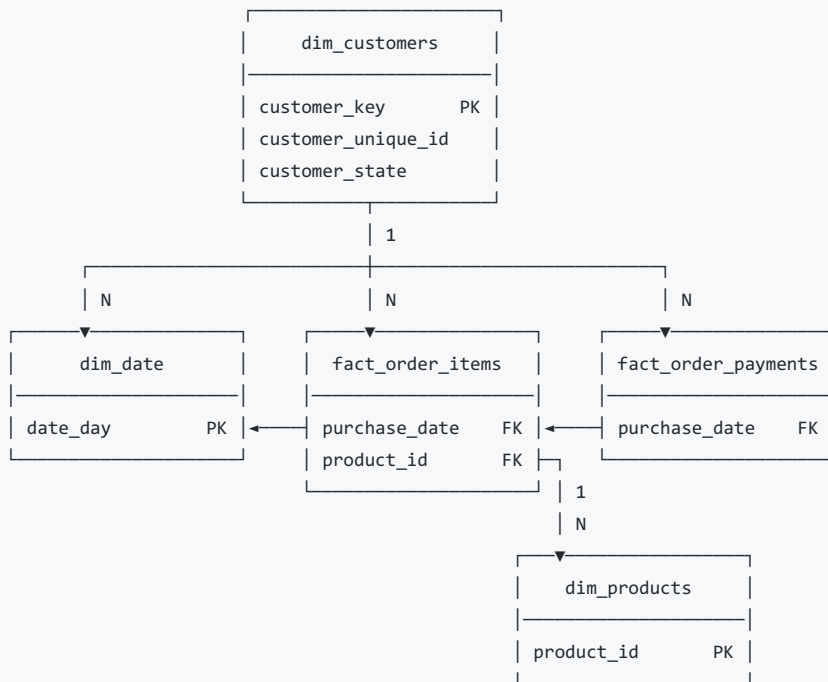
duplication.

- **Dimensions:** Performs full scans of master files to capture updates, updating snapshot timelines.

4.4. Late-Arriving Dimensions Handling

- **Problem:** Facts are refreshed 3 times a day, while dimensions are updated once a day at midnight. A new customer placing an order at 10:00 AM will have transaction records in `fact_order_items` at 16:30 PM, but their customer details will not be available in `dim_customers` until midnight.
- **Database Risk:** Applying strict foreign key constraints at the database layer will cause transaction loading to fail, halting the pipeline.
- **Solution:**
 1. Remove physical database foreign key constraints.
 2. Implement `LEFT JOIN` in fact building to prevent order record loss.
 3. Apply `COALESCE(dim.customer_key, 'pending_refresh')` to assign a temporary placeholder key to late-arriving customers.
 4. Once the daily dimension job runs, the relationships are automatically resolved.

5. Data Model (Star Schema)



5.1. Fact Tables

Table `fact_order_items`

- **Grain:** 1 row = 1 product item per order (`order_id` + `order_item_id`).
- **Columns:** `order_item_key` (PK), `order_id`, `customer_key` (FK SCD Type 2), `product_id` (FK), `purchase_date` (FK), `price_br1`, `freight_br1`, `price_usd`, `freight_usd`, `order_status`.

Table `fact_order_payments`

- **Grain:** 1 row = 1 payment transaction per order (`order_id` + `payment_sequential`).
- **Columns:** `order_payment_key` (PK), `order_id`, `customer_key` (FK), `payment_type`, `payment_installments`, `payment_value_br1`, `payment_value_usd`, `purchase_date`.

5.2. Dimension Tables

- **dim_customers:** Surrogate key `customer_key`. Tracks historical address updates using **SCD Type 2** snapshots (`dbt_valid_from`, `dbt_valid_to`).
- **dim_products:** Primary key `product_id`. Employs **SCD Type 1** to maintain the latest English translation.
- **dim_date:** Primary key `date_day`. Supports MTD calculations and calendar filtering.
- **dim_exchange_rate:** Primary key `date_day`. Stores converted `br1_to_usd_rate` with weekend rates forward-filled.

6. Ingest & Idempotency Strategy

Layer	Idempotency Mechanism
-------	-----------------------

Layer	Idempotency Mechanism
Raw Layer	• Static CSV Mechanism: Truncate + LOAD execution to overwrite tables, preventing row multiplication. • Exchange Rates: Employs UPSERT (ON CONFLICT (date_day) DO UPDATE) to overwrite rates for re-runs.
Warehouse Layer	• Dimensions: dim_products rebuilt fully (table materialization). dim_customers uses dbt snapshot to compare hashes and track changes. • Facts: Materialized as incremental using merge on unique keys (order_item_key / order_payment_key) with a 30-day lookback window to update or insert records.
BI Layer	• Separates Fact tables to prevent fan-out row inflation when aggregating sales and payments.

Idempotency Verification Query

```
-- Step 1: Count rows after execution 1
SELECT (SELECT COUNT(*) FROM fact_order_items) AS r1_items, (SELECT COUNT(*) FROM fact_order_payments) AS r1_payments;

-- Step 2: Re-run the dbt pipeline

-- Step 3: Count rows again (Row counts must match Step 1)
SELECT (SELECT COUNT(*) FROM fact_order_items) AS r2_items, (SELECT COUNT(*) FROM fact_order_payments) AS r2_payments;
```

7. Data Quality

7.1. Automated Tests (dbt Tests)

- **fact_order_items**: unique and not_null constraints on order_item_key; relationships tests verifying keys exist in dim_products and dim_customers (severity set to warn to prevent pipeline blockages during sync delays).
- **fact_order_payments**: unique and not_null constraints on order_payment_key.
- **Exchange Rates**: not_null check on price_usd and payment_value_usd.

7.2. Error Handling & Alerting

- **Missing FX Rates**: Checks for price_usd IS NULL . The pipeline logs a warning if a rate is missing.
- **Orphan Products**: Automatically alerts if product IDs in facts do not map to the dimension.

8. BI Queries

8.1. Total Revenue & MTD (USD)

```
SELECT
    SUM(price_usd) AS total_revenue_usd,
    SUM(CASE WHEN purchase_date >= DATE_TRUNC('month', '2018-08-15'::date) THEN price_usd ELSE 0 END) AS mtd_revenue_usd
FROM fact_order_items
WHERE order_status NOT IN ('canceled', 'unavailable');
```

8.2. Revenue by Product Category

```
SELECT f.product_category, SUM(f.price_usd) AS revenue_usd
FROM public_core.fact_order_items f
WHERE f.order_status NOT IN ('canceled', 'unavailable')
GROUP BY 1 ORDER BY 2 DESC;
```

8.3. Revenue by Customer State (Region)

```
SELECT f.customer_state, SUM(f.price_usd) AS revenue_usd
FROM public_core.fact_order_items f
WHERE f.order_status NOT IN ('canceled', 'unavailable')
GROUP BY 1 ORDER BY 2 DESC;
```

8.4. Freight Contribution Ratio

```
SELECT
    SUM(freight_usd) AS total_freight_usd,
    SUM(price_usd) AS total_revenue_usd,
    SUM(freight_usd) / NULLIF(SUM(price_usd), 0) * 100 AS freight_ratio_pct
FROM fact_order_items
WHERE order_status NOT IN ('canceled', 'unavailable');
```

8.5. Payment Methods & Installment Aggregation

```
SELECT
    payment_type,
    SUM(payment_value_usd) AS total_payment_usd,
    AVG(payment_installments) AS avg_installments
FROM fact_order_payments
GROUP BY 1 ORDER BY 2 DESC;
```

8.6. Lifetime Repeat Buyer Rate

```
WITH customer_order_counts AS (
    SELECT customer_unique_id, COUNT(DISTINCT order_id) AS total_orders
    FROM fact_order_items
    WHERE order_status NOT IN ('canceled', 'unavailable')
    GROUP BY 1
)
SELECT
    COUNT(*) AS total_customers,
    COUNT(CASE WHEN total_orders > 1 THEN 1 END) AS repeat_buyers,
    COUNT(CASE WHEN total_orders > 1 THEN 1 END)::FLOAT / NULLIF(COUNT(*), 0) * 100 AS repeat_buyer_pct
FROM customer_order_counts;
```

8.7. 90-Day Repeat Buyer Rate

```

WITH order_pairs AS (
    SELECT
        a.customer_unique_id,
        a.order_id,
        a.purchase_date AS current_purchase_date,
        MAX(b.purchase_date) AS last_purchase_date
    FROM fact_order_items a
    LEFT JOIN fact_order_items b
        ON a.customer_unique_id = b.customer_unique_id
        AND b.purchase_date < a.purchase_date
        AND b.order_status NOT IN ('canceled', 'unavailable')
    WHERE a.order_status NOT IN ('canceled', 'unavailable')
    GROUP BY 1, 2, 3
),
labeled_orders AS (
    SELECT
        customer_unique_id,
        CASE WHEN last_purchase_date IS NOT NULL AND (current_purchase_date - last_purchase_date) <= 90 THEN 1 ELSE 0 END AS is_90d_repeat
    FROM order_pairs
)
SELECT
    COUNT(DISTINCT customer_unique_id) AS total_customers,
    COUNT(DISTINCT CASE WHEN is_90d_repeat = 1 THEN customer_unique_id END) AS repeat_buyers_90d,
    COUNT(DISTINCT CASE WHEN is_90d_repeat = 1 THEN customer_unique_id END)::FLOAT
    / NULLIF(COUNT(DISTINCT customer_unique_id), 0) * 100 AS repeat_rate_90d_pct
FROM labeled_orders;

```

Note: The 90-day repeat interval is a design assumption — documented in Section 9.1.

9. Design Dilemmas, Uncertainties & Limitations

9.1. Design Dilemmas

- **Revenue Recognition at approved Status:** I chose to calculate revenue as soon as an order is marked as `approved`. This provides the sales team with immediate feedback on the dashboard, but introduces a risk: if an order is canceled after 30 days, historical monthly revenue figures will retroactively decrease, causing discrepancies against fixed monthly financial reports.
- **Handling Installment Cash Flow:** Installment payments spread cash inflows over multiple months. However, I assumed 100% of the transaction revenue is recognized immediately on the purchase date. This focuses the dashboard on sales performance (KPI Sales) rather than cash flow tracking.
- **Revenue: Price vs Payment Value:** I chose price in `order_items` over `payment_value` in `order_payments` to allow revenue breakdown by Product Category. The trade-off is that if an order has a platform discount or payment fee discrepancy, the price-derived revenue will slightly deviate from actual cash collected.
- **Excluding Geolocation Details:** The geolocation table has duplicate zip code mappings, creating a Many-to-Many relationship that leads to row inflation. I chose to exclude this table from the MVP and rely on `customer_state` for geographical reports, prioritizing data integrity over detailed GPS plotting.
- **SCD Type 1 vs SCD Type 2 for Customers:** I implemented SCD Type 2 using dbt snapshots to track customer address history. This ensures that past revenue is attributed to the region where the customer lived at the time of purchase, accepting the trade-off of more complex timestamp-based SQL JOINS (`BETWEEN valid_from AND valid_to`) in fact tables.
- **90-Day Repeat Buyer Rate Assumption:** The dataset does not define a returning buyer. I assumed a 90-day window between purchases to measure retention. While appropriate for FMCG, this window is too short for high-ticket appliances:
 - Fast-moving categories (beauty, fashion): Repurchase within 30-90 days is common.
 - Slow-moving categories (appliances, electronics): Repurchase cycles span 1-2 years. Therefore, the 90-day threshold is a simplified general assumption for this challenge.
- **Lookback Window for Late Updates:** Due to the lack of an `updated_at` column in the source data, I applied a **30-day lookback window** in dbt incremental models. This assumes order updates complete within 30 days of purchase. If an order update takes longer, it will be missed by incremental runs.

9.2. Data Uncertainties

[!NOTE] **SCD Type 2 Technical Limitation on Static Initial Load:** Because the historical data is loaded from static CSV files, the system deduplicates raw customer records (taking the last known state via `rn = 1`) before loading them into the dbt snapshot to prevent primary key duplicates. Consequently, historical customer data effectively functions as SCD Type 1. However, the SCD Type 2 infrastructure is established and will automatically track address changes once new incremental data is loaded daily via Airflow.

- **Vouchers & Platform Promotions:** I assumed all vouchers are logged as payment methods to balance total payments with invoice totals. In practice, if a discount is subtracted directly from the cart without being recorded in the payment gateway, price-derived revenue will be higher than actual cash receipts.
- **Weekend Exchange Rates:** Although the code forward-fills the Friday exchange rate to Saturday and Sunday, the long-term availability of the free Frankfurter API is an external dependency.
- **Missing Returns/Refunds Data:** The Olist dataset does not record returns. I assumed all `delivered` orders are permanently successful.

9.3. Project Limitations

- **Domain Knowledge Limitations:** Because my expertise in accounting and corporate cash flow management is limited, payment installment assumptions and revenue recognition at order approval may not align with strict accounting standards. The project focuses purely on sales metrics (KPI Sales).
- **Static Data Source Limitations:** The ingestion pipeline reads from static CSV files rather than staging CDC/streaming replication from a live production database. However, the dbt core models are fully designed to support incremental loading when live data sources become available.
- **Airflow Monitoring Limits:** In this local Docker deployment, there are no centralized logging tools or automated alerts (e.g., Slack/Telegram/Email notifications) to notify administrators of pipeline failures.
- **Data Testing Limits:** The dbt testing suite currently covers standard constraints (uniqueness, non-null values, relationships, and revenue reconciliation). A production system would require more advanced integration testing.
- **Omitted Tables in MVP (Reviews and Geolocation):**
 - *Reviews (order_reviews):* Contains duplicate reviews per order, requiring complex deduplication, so I excluded it to prioritize core sales metrics.
 - *Geolocation (geo_location):* Excluded to prevent fan-out row inflation, using `customer_state` instead.

10. Scale-up Plan (Handling Millions of Rows)

- If the dataset scales to hundreds of millions of rows, the current local PostgreSQL setup will become a bottleneck. I would implement the following architecture:
1. **Columnar Storage (Parquet format):** Store raw files on cloud storage in Parquet format. Columnar format allows queries to scan only the necessary columns (e.g., aggregating `price` without scanning other fields), reducing I/O costs.
 2. **Cloud Data Warehousing (BigQuery / Snowflake):** Migrate from Postgres to BigQuery or Snowflake. These platforms separate compute and storage, distributing query tasks across compute clusters.
 3. **Partitioning & Clustering:**
 - **Partitioning by Date:** Partition fact tables by *order purchase month*. Queries filtering for specific months will scan only that partition, reducing query costs.
 - **Clustering by Keys:** Cluster tables by frequently filtered columns (e.g., `product_id`, `customer_state`) to physically group matching records together, accelerating query speeds.
 4. **Incremental dbt Builds:** Maintain the current dbt incremental setup, ensuring daily runs only process current delta records rather than executing full refreshes.
 5. **Production Airflow:** Deploy Airflow on a managed service (e.g., Astronomer or Cloud Composer) to support parallel processing and automated error alerting.
 6. **One Big Table (OBT) for the Reporting Layer:** For the BI Mart layer, denormalize the Star Schema into One Big Table (OBT) via dbt. While increasing storage consumption, it eliminates costly runtime `JOIN` operations, ensuring sub-second dashboard rendering.

Appendix – Assumption Registry

#	Assumption	Business Rationale	Verification Status
A1	Revenue = <code>order_items.price</code> (excluding freight)	<code>order_payments</code> is at the order level. Only <code>order_items</code> provides the item grain needed for product category revenue splits.	Verified – query 3.3
A2	Order date = <code>order_purchase_timestamp</code>	Represents the timestamp of customer checkout commitment.	Verified
A3	Exchange rate fetched on purchase date	Reflects the currency value at checkout.	Verified
A4	Weekend rates use Friday's rate (forward-fill)	Financial markets close on weekends.	Verified
A5	Repeat buyers calculated using <code>customer_unique_id</code>	<code>customer_id</code> is unique per transaction.	Verified – query 3.1
A6	<code>customer_state</code> represents Region	Geolocation creates fan-out row inflation.	Verified – query 3.5
A7	Exclude <code>canceled</code> and <code>unavailable</code> statuses	Excludes unrealized revenue from KPI metrics.	Design choice
A8	Prices include consumer tax (Gross Sales)	Standard in Brazilian ICMS taxation.	Verified
A9	<code>payment_type = 'not_defined'</code> mapped to 'Unknown'	Prevents dropping records, preserving total payments.	Verified
A10	<code>delivered</code> status orders are never returned	No returns or refunds tables are available.	Verified
A11	Reports both Lifetime and 90-Day Repeat rates	Balances historical lifetime accumulation with active engagement metrics.	Verified

#	Assumption	Business Rationale	Verification Status
A12	Uses a 30-day lookback window for incremental runs	Assumes most orders finalize state transitions within 30 days.	Verified